STATS 413 - Lecture 24

Bayesian Normal Regression

Introduction

- In this lecture we introduce the normal linear regression model from a Bayesian perspective
- A dataset will be given and a problem will be described
- Noninformative priors will be used on all model parameters
- This will allow us to compare the standard linear model with the Bayesian one
- We will fit the standard model in a non-Bayesian framework and then compare and contrast the results with those obtained using WinBUGS and R-INLA
- We should be able to see the advantages and disadvantages of the various approaches

Data Problem - Soft Drink Delivery Times

Description of the problem:

A soft drink company is interested in the quality of their delivery system. We are interested in estimation of the time required by each employee to refill an automatic vending machine owned and served by the company. A data analyst was hired by the company to set up a small quality assurance study. As the response variable, the analyst considered total service time of each machine measured in minutes, which included stocking each machine with beverage products and any required maintenance or other service.

After examining the problem, the analyst recommended two important variables that affect delivery time: the number of cases of stocked products and the distance walked by the employee measured in feet.

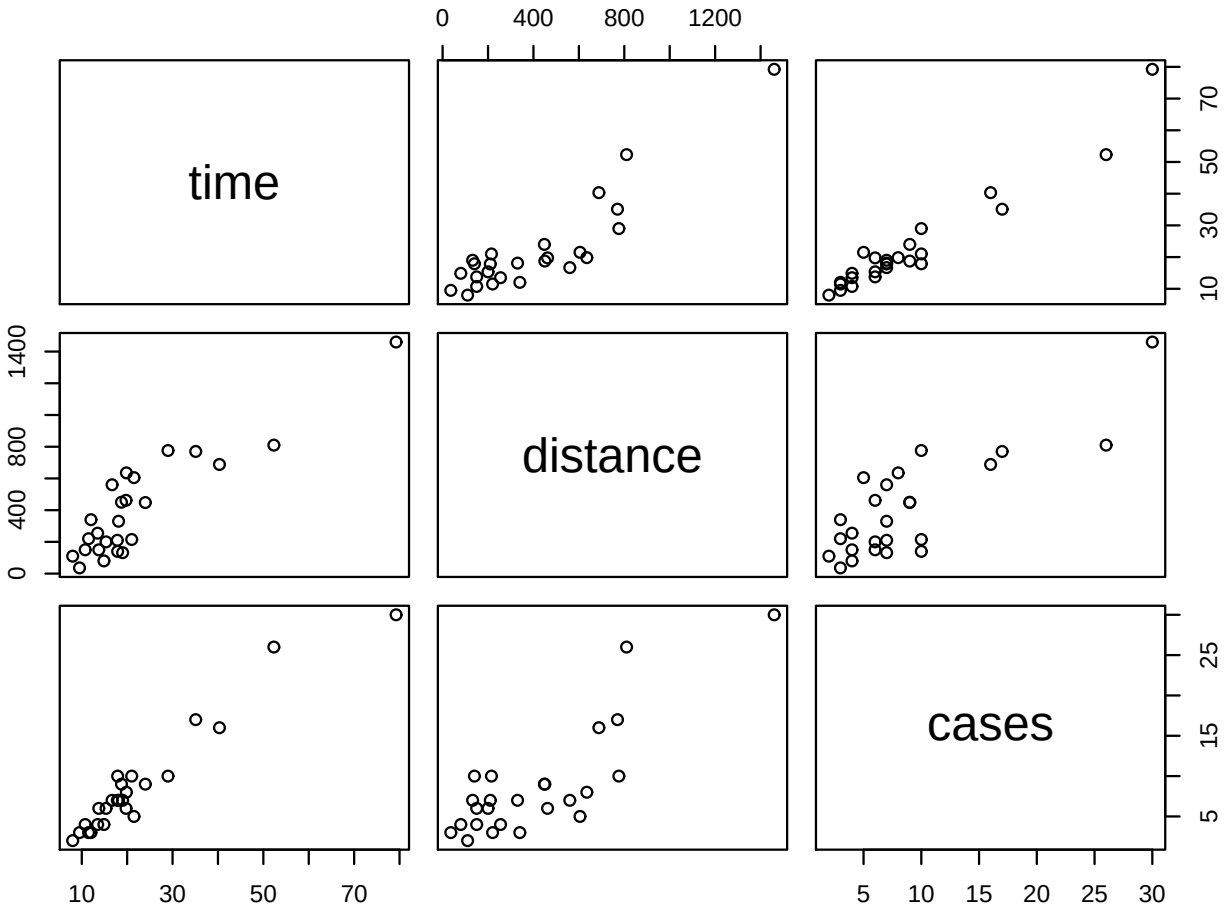
A dataset of 25 observations was collected and is given on the next slide.

Data - Service Time of Soft Drink Machines

Soft Drink Service Time Data

```
time <- c(16.68, 11.5, 12.03, 14.88, 13.75, 18.11,  8, 17.83,
          79.24, 21.5, 40.33, 21, 13.5, 19.75, 24, 29, 15.35,
          19, 9.5, 35.1, 17.9, 52.32, 18.75, 19.83, 10.75)
distance <- c(560, 220, 340, 80, 150, 330, 110, 210, 1460,
              605, 688, 215, 255, 462, 448, 776, 200, 132,
              36, 770, 140, 810, 450, 635, 150)
cases <- c( 7, 3, 3, 4, 6, 7, 2, 7, 30, 5, 16, 10, 4, 6, 9,
           10, 6, 7, 3, 17, 10, 26, 9, 8, 4)

servicedat <- data.frame(time=time,distance=distance,cases=cases)

pairs(servicedat)
```

Matrix Scatterplot - Soft Drink Data



The Linear Model

The standard model is

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \epsilon_i \qquad \epsilon_i \overset{\text{indep}}{\sim} N(0, \sigma^2) \qquad i = 1, \ldots, 25$$

or

$$y_i \sim N(\mu_i, \sigma^2) \qquad y_i \text{ independent} \qquad i = 1, \ldots, 25$$

$$\mu_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2}$$

where

$$y = \text{time}$$
$$x_1 = \text{cases}$$
$$x_2 = \text{distance}$$

Fit The Standard Linear Model - Results

- Results of the model fit are shown below
- Estimates of β_0, β_1, β_2, σ, and R^2 are shown

```
m1 <- lm(time ~ cases + distance, data=servicedat)
summary(m1)

Coefficients:
            Estimate Std. Error t value Pr(>|t|)
(Intercept) 2.341231   1.096730   2.135 0.044170 *
cases       1.615907   0.170735   9.464 3.25e-09 ***
distance    0.014385   0.003613   3.981 0.000631 ***
---
Residual standard error: 3.259 on 22 degrees of freedom
Multiple R-squared:  0.9596,    Adjusted R-squared:  0.9559
F-statistic: 261.2 on 2 and 22 DF,  p-value: 4.687e-16
```

Confidence and Prediction Intervals

- A confidence interval for the mean response and a prediction interval for a new y are shown below
- The predictors x_1 (cases) and x_2 (distance) are set at their mean values, so 22.38 represents a *typical* service time in some sense

```
mean_cases <- mean(cases)
mean_dist <- mean(distance)
dat_frame <- data.frame(cases=mean_cases,distance=mean_dist)
predict(m1,dat_frame,interval="confidence",se=TRUE)
predict(m1,dat_frame,interval="prediction")
```

```
     fit      lwr      upr
1 22.384 21.03205 23.73595
     fit     lwr       upr
1 22.384 15.4904   29.2776
```

Bayesian Regression Using WinBUGS

Bayesian Regression Using WinBUGS

- We now use WinBUGS and consider a Bayesian model with noninformative priors
- By using noninformative priors, results should be similar to those just obtained using R
- A Bayesian version of Adjusted R^2 will be calculated
- A Bayesian confidence interval for the mean response, and a Bayesian prediction interval for a new y will be calculated
- We will run Markov chains with 50,000 iterations, and discard the first 5,000 for burn-in

A Bayesian Model With Noninformative Priors

The model for the soft drink data is

Likelihood:

$$y_i|\beta_0, \beta_1, \beta_2, \tau \sim N(\mu_i, \sigma^2 = \frac{1}{\tau}) \qquad y_i|\beta_0, \beta_1, \beta_2, \tau \text{ independent}$$

$$\mu_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} \qquad i = 1, \ldots, 25$$

Noninformative Prior:

$$\beta_j \sim N(0, 10^4) \quad j = 0, 1, 2 \qquad \tau \sim \text{Gamma}(0.01, 0.01)$$

WinBUGS code is given on the next two slides

WinBUGS Code - Soft Drink Example

```
model{
   # likelihood
   for (i in 1:n){
        time[i] ~ dnorm( mu[i], tau ) # response normal
        # regression function
        mu[i] <- beta0 + beta1 * cases[i] + beta2 * distance[i]
   }
   # prior distributions
   tau ~ dgamma( 0.01, 0.01 )
   beta0 ~ dnorm( 0.0, 1.0E-4)
   beta1 ~ dnorm( 0.0, 1.0E-4)
   beta2 ~ dnorm( 0.0, 1.0E-4)
   # definition of sigma
   s2<-1/tau
   s <-sqrt(s2)
   # calculation of the sample variance
   for (i in 1:n){ c.time[i]<-time[i]-mean(time[]) }
   sy2 <- inprod( c.time[], c.time[] )/(n-1)
   # calculation of Bayesian version Adj R squared
   R2B <- 1 - s2/sy2
   # Bayesian estimate and confidence interval for mean response
   # at the means of predictors
   newy <- beta0 + beta1 * mean(cases[]) + beta2 * mean(distance[])
   # Bayesian estimate and prediction interval for new y
   # at the means of predictors
   ynew ~ dnorm(newy,tau)
}
```

WinBUGS Code - Soft Drink Example (cont)

```
INITS
list( tau=1, beta0=0, beta1=0, beta2=0 )

DATA (LIST)
list( n=25,
      time = c(16.68, 11.5, 12.03, 14.88, 13.75, 18.11,  8, 17.83,
               79.24, 21.5, 40.33, 21, 13.5, 19.75, 24, 29, 15.35,
               19, 9.5, 35.1, 17.9, 52.32, 18.75, 19.83, 10.75),
      distance = c(560, 220, 340, 80, 150, 330, 110, 210, 1460,
                   605, 688, 215, 255, 462, 448, 776, 200, 132,
                   36, 770, 140, 810, 450, 635, 150),
      cases = c( 7, 3, 3, 4, 6, 7, 2, 7, 30, 5, 16, 10, 4, 6, 9,
                 10, 6, 7, 3, 17, 10, 26, 9, 8, 4) )
```

Data may also be input in rectangular format as shown on the following slide

WinBUGS Code - Soft Drink Example (cont)

```
    DATA (RECT.)
    list(n=25)
    time[] cases[] distance[]
    16.68    7     560
    11.5     3     220
    12.03    3     340
    14.88    4      80
    13.75    6     150
    18.11    7     330
     8       2     110
    17.83    7     210
    79.24   30    1460
    21.5     5     605
    40.33   16     688
    21      10     215
    13.5     4     255
    19.75    6     462
    24       9     448
    29      10     776
    15.35    6     200
    19       7     132
     9.5     3      36
    35.1    17     770
    17.9    10     140
    52.32   26     810
    18.75    9     450
    19.83    8     635
    10.75    4     150
END
```

Bayesian Estimation - Results

- Bayesian estimates of β_0, β_1, β_2, σ, and R^2 are shown below

Node statistics

node	mean	sd	MC error	2.5%	median	97.5%	start	sample
beta0	2.338	1.152	0.00547	0.08217	2.339	4.619	5001	45000

Node statistics

node	mean	sd	MC error	2.5%	median	97.5%	start	sample
beta1	1.616	0.1792	8.673E-4	1.261	1.616	1.969	5001	45000

Node statistics

node	mean	sd	MC error	2.5%	median	97.5%	start	sample
beta2	0.01438	0.003799	1.843E-5	0.00689	0.01437	0.02195	5001	45000

Node statistics

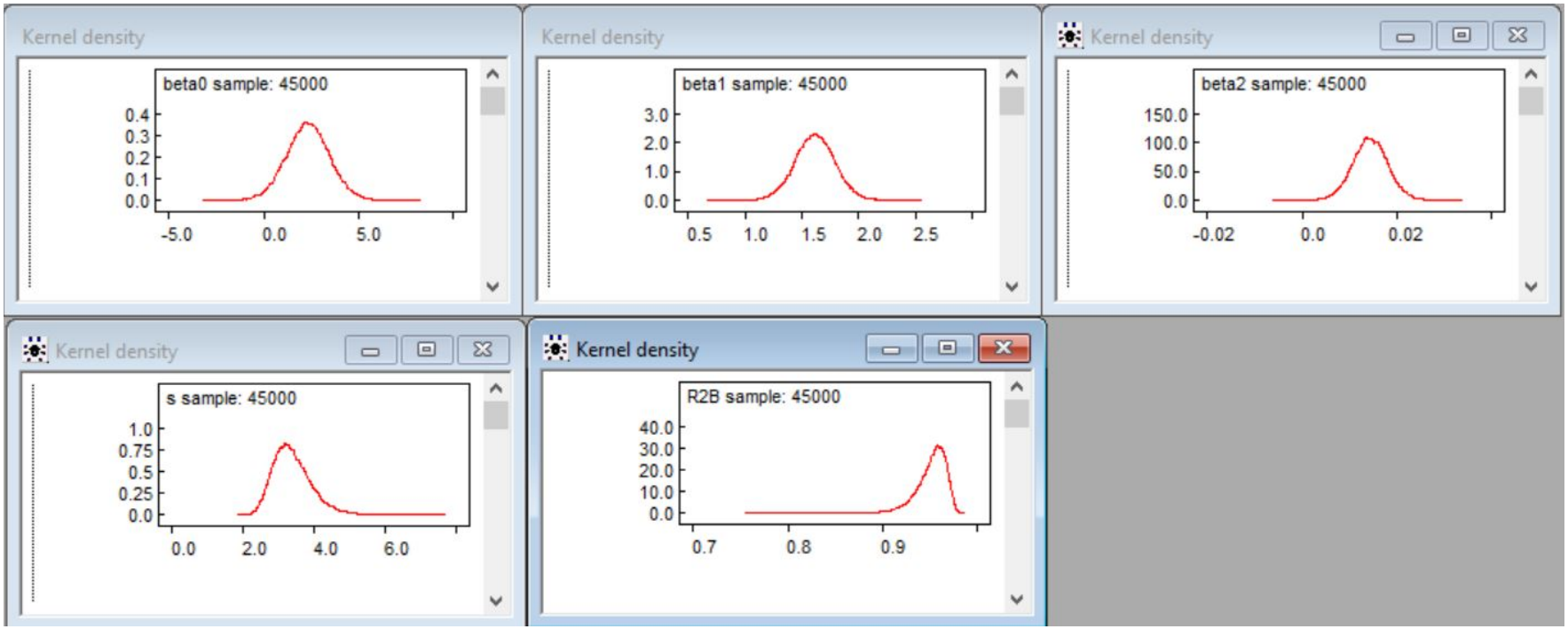
node	mean	sd	MC error	2.5%	median	97.5%	start	sample
s	3.378	0.5393	0.002797	2.518	3.309	4.616	5001	45000

Node statistics

node	mean	sd	MC error	2.5%	median	97.5%	start	sample
R2B	0.9515	0.01623	8.449E-5	0.9116	0.9546	0.9737	5001	45000

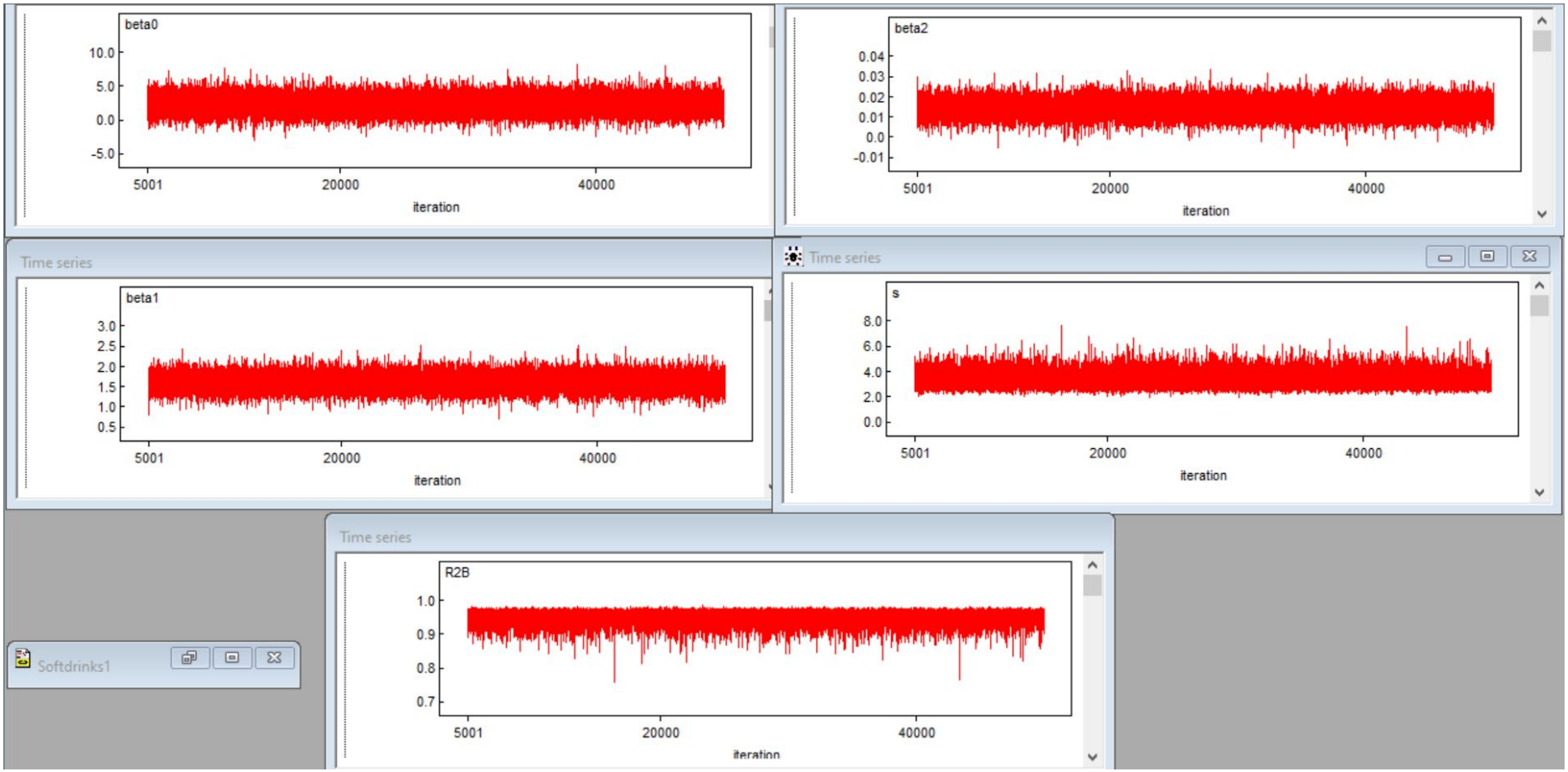
Bayesian Estimation - Results

- Corresponding density plots of β_0, β_1, β_2, σ, and R^2 are shown below



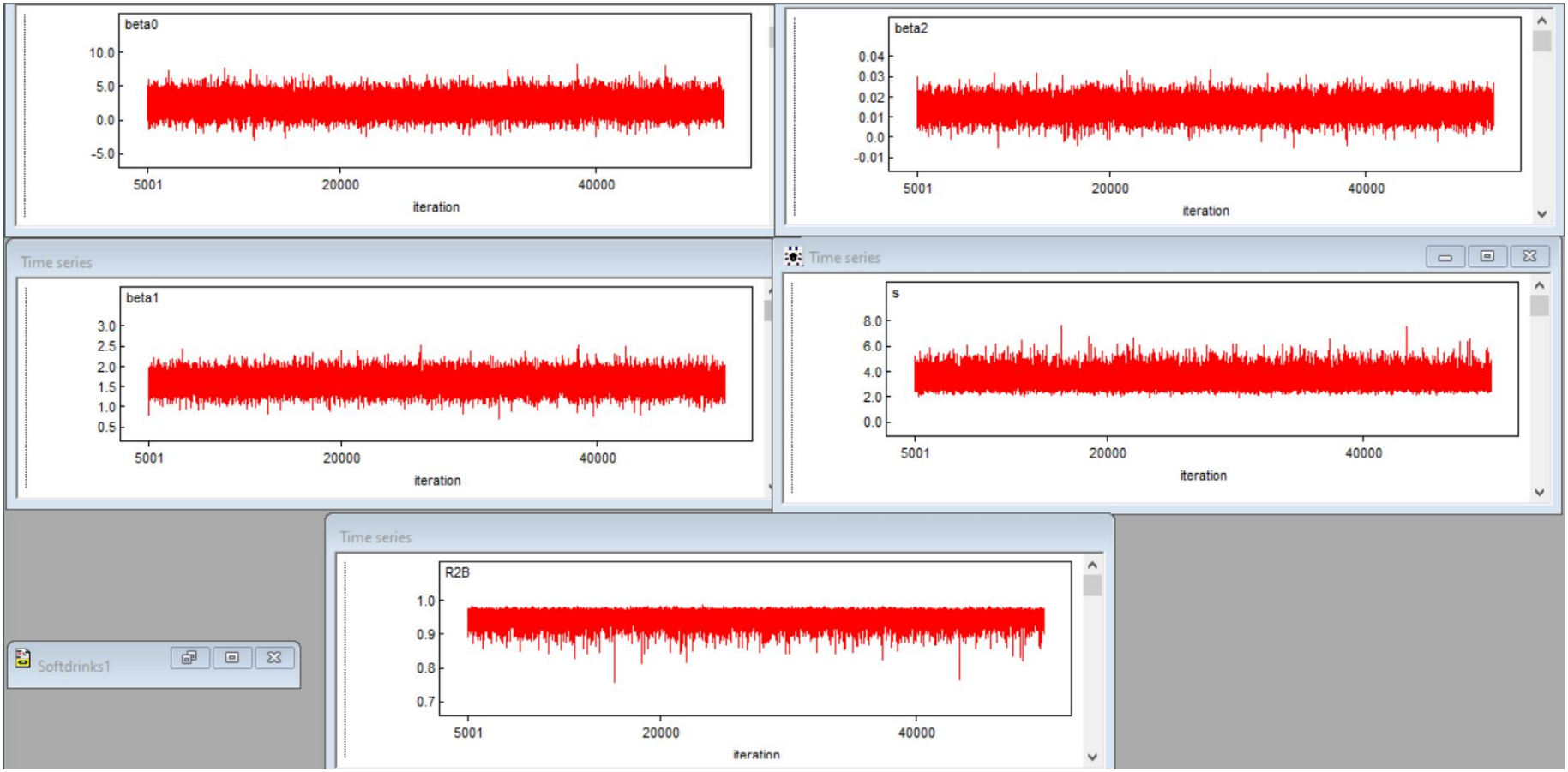
Bayesian Estimation - Results

- Corresponding time series plots of β_0, β_1, β_2, σ, and R^2 simulations to check for convergence and mixing of the chains



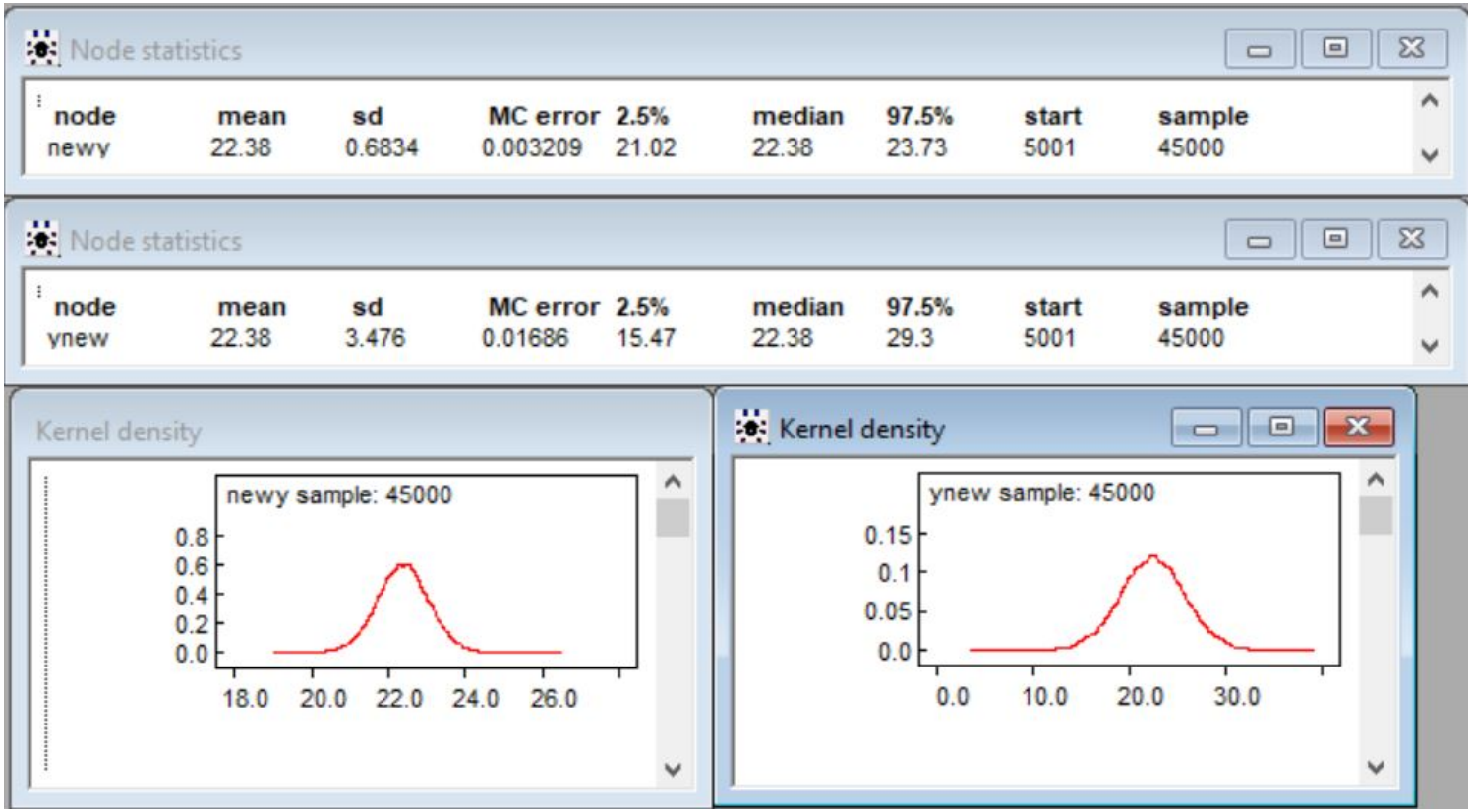
Bayesian Estimation - Results

- Corresponding time series plots of β_0, β_1, β_2, σ, and R^2 simulations to check for convergence and mixing of the chains



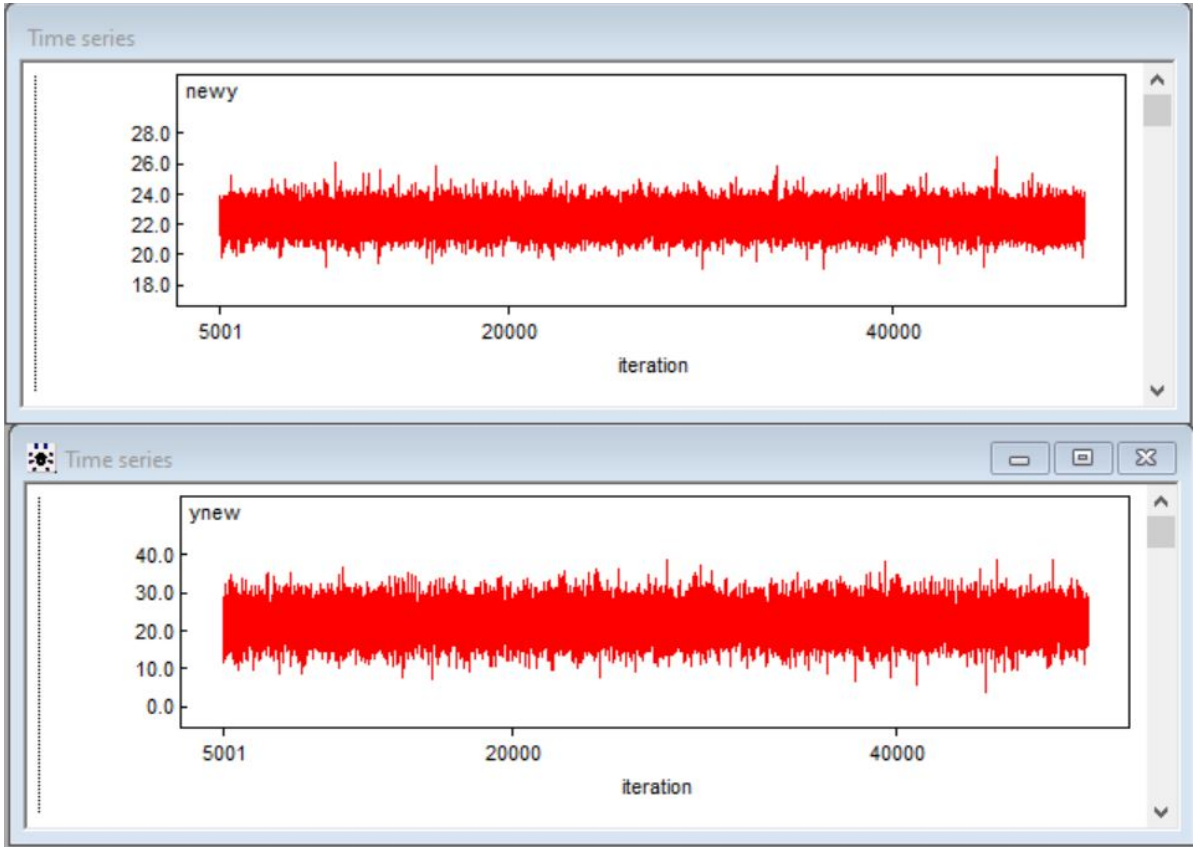
Bayesian Confidence and Prediction Interval Results

- Bayesian confidence interval for the mean response and a prediction interval for a new y



Bayesian Confidence and Prediction Interval Results

- Time series plots for the mean response and prediction interval to check for convergence and mixing of the chains



Bayesian Regression Using R-INLA

Bayesian Regression Using R-INLA

- We now use R-INLA and consider the same Bayesian model outlined in Slide 12
- Note that INLA uses analytic approximations and Markov chain Monte Carlo sampling is not perfomed
- One of the major advantages is that computing time is significantly reduced relative to the simulation-based approach
- This can be an issue when analyzing large data sets, or when considering more complicated models

The R-INLA Code

- The R-INLA code is shown below

```
library(INLA)
formula <- time ~ 1 + cases + distance

m2 <- inla(formula,family="gaussian",data=servicedat,
        control.fixed=list(mean=0,prec=0.0001,
        mean.intercept=0,prec.intercept=0.0001),
        control.family=list(hyper=list(
        prec=list(prior="loggamma",param=c(0.01,0.01)))))

summary(m2)
```

Bayesian Estimation - Results

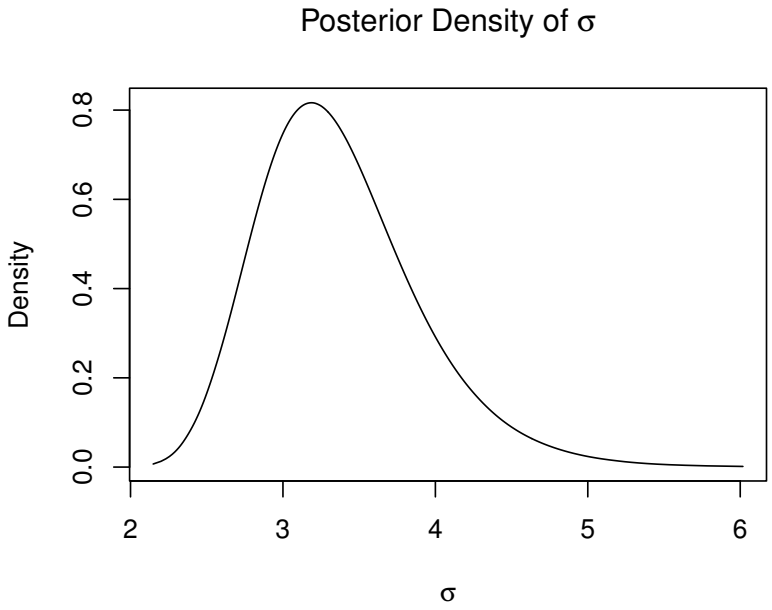
- Estimates of β_0, β_1, β_2, and the precision parameter $\tau = 1/\sigma^2$

```
Fixed effects:
             mean    sd 0.025quant 0.5quant 0.975quant  mode
(Intercept) 2.341 1.144      0.073    2.341      4.607 2.341
cases       1.616 0.178      1.263    1.616      1.969 1.616
distance    0.014 0.004      0.007    0.014      0.022 0.014

Model hyperparameters:
Precision for the         mean    sd 0.025quant 0.5quant 0.975quant  mode
Gaussian observations    0.094 0.028      0.047    0.091      0.157 0.086
```

Making A Transformation

- If we want to estimate σ instead of τ, we can use the `inla.tmarginal` function to make a transformation

```
prec.post <- m2$marginals.hyper[[1]]
s.post <- inla.tmarginal(function(x) 1/sqrt(x),prec.post)
plot(s.post,type="l",xlab=expression(sigma),ylab="Density",
    main=expression(paste("Posterior Density of ",sigma)))
inla.zmarginal(s.post)
```

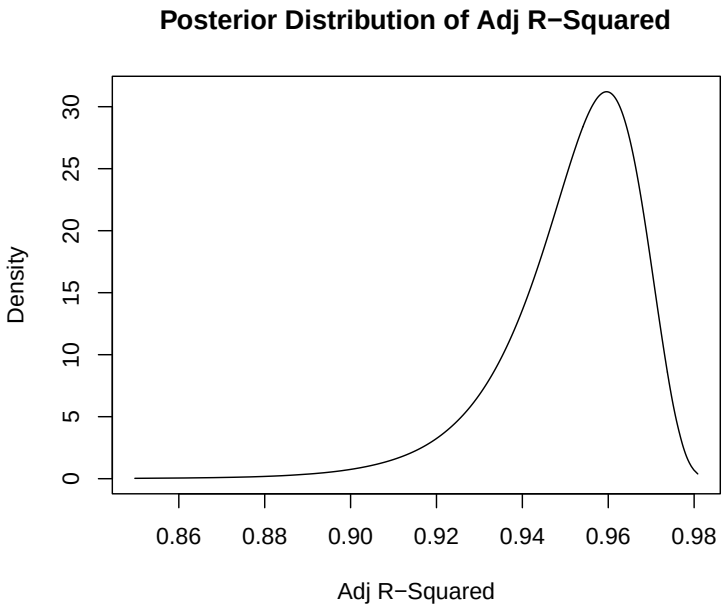


Mean		3.37352
Stdev		0.53124
Quantile	0.025	2.52094
Quantile	0.5	3.30729
Quantile	0.975	4.60173

Bayesian Version of Adjusted R-Squared

- The `inla.tmarginal` function can also be used to estimate adjusted R-Squared

```
sy2 <- var(servicedat$time)
R2B <- inla.tmarginal(function(x) 1-1/(x*sy2),prec.post)
plot(R2B,type="l", xlab="Adj R-Squared",ylab="Density",
   main="Posterior Distribution of Adj R-Squared")
inla.zmarginal(R2B)
```




```
Mean            0.951611
Stdev           0.015869
Quantile  0.025 0.912073
Quantile  0.5   0.95456
Quantile  0.975 0.973581
```

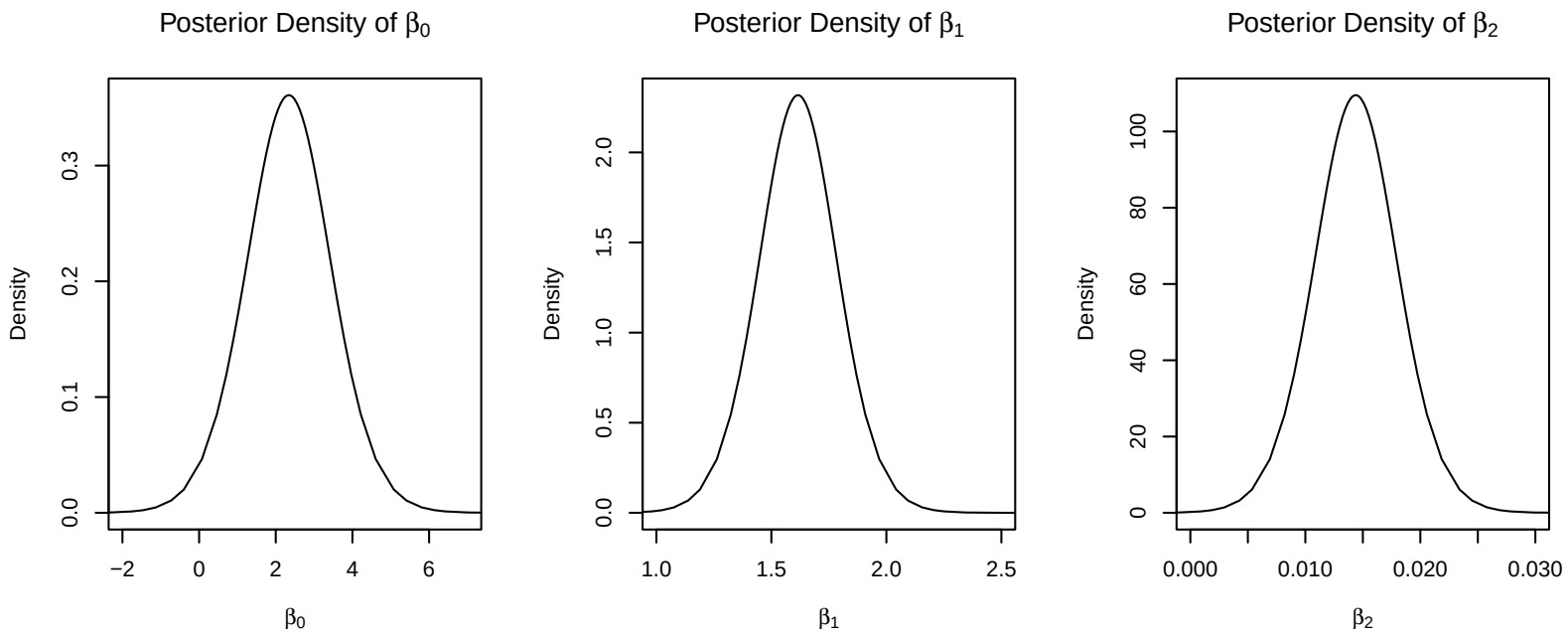
The Regression Coefficients

- Code to make plots of the posterior densities of the regression coefficients is given below

```
names(m2$marginals.fixed)
beta0 <- m2$marginals.fixed[[1]]
beta1 <- m2$marginals.fixed[[2]]
beta2 <- m2$marginals.fixed[[3]]

par(mfrow=c(1,3))
plot(beta0,type="l",xlab=expression(beta[0]),ylab="Density",
  xlim=c(-2,7),main=expression(paste("Posterior Density of ",beta[0])))
plot(beta1,type="l",xlab=expression(beta[1]),ylab="Density",
  xlim=c(1,2.5),main=expression(paste("Posterior Density of ",beta[1])))
plot(beta2,type="l",xlab=expression(beta[2]),ylab="Density",
  xlim=c(0,0.03),main=expression(paste("Posterior Density of ",beta[2])))
```

The Regression Coefficients

- Plots of β_0, β_1, and β_2



Confidence Interval For the Mean Response

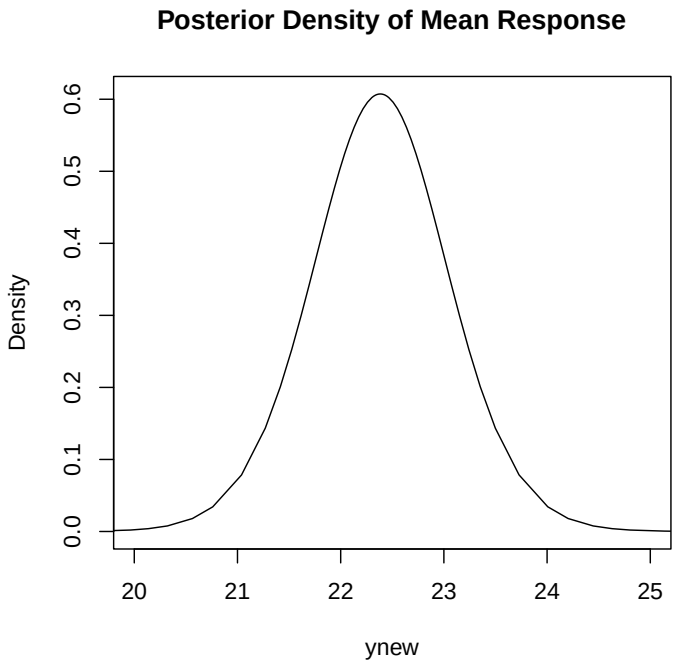
- Code for a confidence interval for the mean response at the mean values of the predictors

```
mean_cases <- mean(cases)
mean_dist <- mean(distance)
newobs <- data.frame(time=NA,distance=mean_dist,cases=mean_cases)
servpred <- rbind(servicedat,newobs)

respred <- inla(formula,family="gaussian",data=servpred,
        control.predictor=list(compute=TRUE),
        control.compute=list(return.marginals.predictor=TRUE),
        control.fixed=list(mean=0,prec=0.0001,
        mean.intercept=0,prec.intercept=0.0001),
        control.family=list(hyper=list(
        prec=list(prior="loggamma",param=c(0.01,0.01)))))

respred$summary.fitted.values[26,]
confint_post <- respred$marginals.fitted.values$fitted.Predictor.26
plot(confint_post,type="l",xlim=c(20,25),xlab="ynew",ylab="Density",
  main="Posterior Density of Mean Response")
```

Confidence Interval For The Mean Response

- Estimates and plot for a confidence interval for the mean response

```
                        mean        sd 0.025quant 0.5quant 0.975quant    mode
fitted.Predictor.26 22.38389 0.6794535   21.03906 22.38389   23.72821 22.3839
```

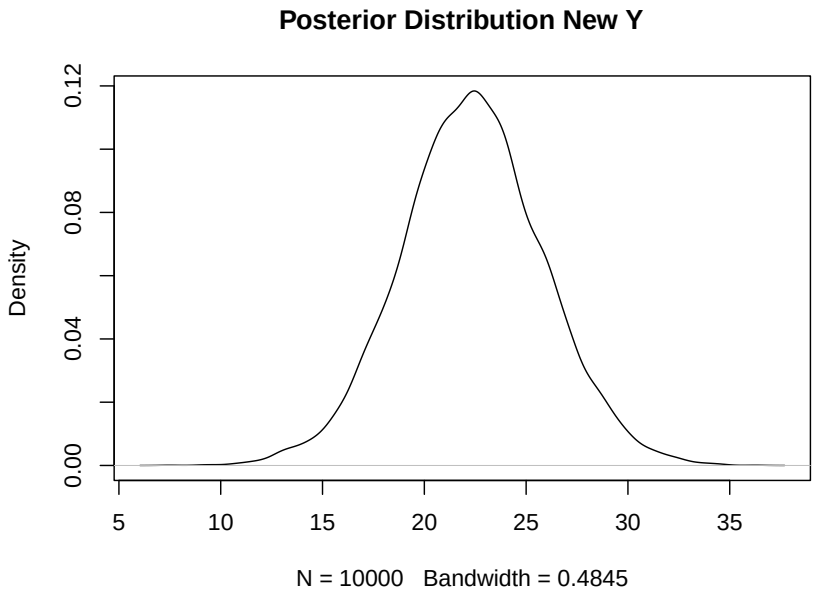


Prediction Of A New Y

- There is not a straightforward way to estimate the posterior predictive distribution using R-INLA
- To get an estimate, we can *trick* INLA by drawing samples from two posterior distributions
- We can draw a sample from the posterior distribution of the mean response (see the previous slide) and we can draw a sample from the posterior distribution of σ (see Slide 26)
- A sample from the posterior predictive distribution can be approximated by adding the sample from the mean response with a sample of normal random numbers with mean 0, and standard deviations drawn from the posterior of σ
- From the sample, the mean, standard deviation, quantiles, and kernel density plot can be made

Prediction Of A New Y

- The function `inla.rmarginal` can be used to draw random numbers from posterior distributions

```
set.seed(123)
s.sample <- inla.rmarginal(10000,s.post)
ci.sample <- inla.rmarginal(10000,confint_post)
post_pred <- ci.sample + rnorm(10000,mean=0,sd=s.sample)
c(mean(post_pred),sd(post_pred),quantile(post_pred,c(.025,.5,.975)))
plot(density(post_pred),main="Posterior Distribution New Y")
```

```
    mean       sd     2.5%      50%    97.5%
22.33913  3.49011 15.46469 22.33404 29.26128
```